Les Structures Répétitives en Algorithmique et Langage C : Boucles While, Do-While et For

Thème principal : Structures itératives (boucles while, do-while, for), conditions d'arrêt et prévention des boucles infinies en C

Synthèse exécutive : Ce cours détaille le fonctionnement, la syntaxe et les critères de choix des trois structures itératives fondamentales (while, do-while, for) en algorithmique et en langage C. L'objectif principal est d'automatiser le traitement répétitif d'instructions sans duplication de code, conformément au principe DRY (Don't Repeat Yourself). Un accent particulier est mis sur l'évaluation des conditions (pré-condition vs post-condition), la gestion des variables de contrôle et la prévention des boucles infinies.

1. Utilité des structures itératives et principe DRY

En algorithmique et en langage C, la répétition d'instructions ne doit jamais être gérée par copier-coller. Si l'on doit traiter 10 000 éléments d'un tableau ou redemander une saisie utilisateur, la duplication manuelle du code rend le programme illisible, impossible à maintenir et génère une source majeure de bogues. Les structures itératives (ou boucles) permettent de faire exécuter un même bloc d'instructions de façon répétée tant qu'une condition logique (expression booléenne) reste vraie. L'un des risques majeurs associés est la 'boucle infinie' : si le bloc d'instructions ne modifie jamais les variables intervenant dans la condition d'arrêt, le programme se bloque indéfiniment en consommant les ressources système.

- Principe DRY (Don't Repeat Yourself) : interdiction fondamentale de dupliquer du code pour répéter une tâche.
- Condition d'arrêt : expression booléenne déterminant la poursuite ou la sortie de la boucle.
- Variable de contrôle / Compteur : variable devant être impérativement mise à jour au fil des itérations pour garantir la fin de la boucle.

Cas concret enseignant : *Métaphore du tapis roulant d'usine d'emballage : tant qu'un colis est détecté par le capteur (condition vraie), le bras robotisé l'emballe (instruction). Si le capteur ne détecte plus de colis ou si le quota est atteint, le robot s'arrête. Sans condition de fin fonctionnelle, le robot continuerait à tourner à vide et détruirait la machine.*

2. La boucle TANT QUE (while) : Structure à pré-condition

La structure 'while' est utilisée lorsque le nombre d'itérations n'est pas déterminé à l'avance, mais dépend d'un événement dynamique externe (saisie utilisateur, lecture de fichier, signal réseau). Le test logique s'effectuant en amont, si la condition est fausse dès la première évaluation, le corps de la boucle est totalement ignoré (0 itération). Pour éviter une boucle infinie, il est impératif que le corps de la boucle modifie au moins une variable impliquée dans l'expression booléenne d'arrêt.

- Pré-conditionnement : la condition d'accès est évaluée AVANT chaque passage dans le bloc d'instructions.
- Intervalle d'itérations (0 à N) : le bloc peut ne jamais s'exécuter si la condition est fausse d'entrée.
- Initialisation préalable : toute variable testée dans la condition doit posséder une valeur définie avant l'instruction while.

Cas concret enseignant : *Exemple du contrôle de code PIN au distributeur de billets : le distributeur demande le code secret TANT QUE le code saisi est incorrect ET QUE le compteur d'essais est inférieur à 3. Si l'usager tape le bon code du premier coup, la boucle s'arrête immédiatement.*

3. La boucle FAIRE... TANT QUE (do-while) : Structure à post-condition

À la différence du 'while', la structure 'do-while' garantit que le corps de la boucle est exécuté au moins une fois avant le premier contrôle de condition. Elle est principalement appliquée aux contrôles de saisie (s'assurer qu'une

valeur numérique est comprise dans un intervalle valide) et à la gestion des menus interactifs. En langage C, sa syntaxe s'écrit `do { ... } while (condition);`. L'oubli du point-virgule final constitue l'erreur de syntaxe la plus fréquente et bloque la compilation.

- Post-conditionnement : le test logique est réalisé APRÈS l'exécution du bloc d'instructions.
- Garantie d'exécution (1 à N) : la boucle s'exécute obligatoirement au minimum 1 fois.
- Syntaxe C stricte : obligation impérative d'apposer un point-virgule ';' après la parenthèse fermante du while.

Cas concret enseignant : *Cas du contrôle de saisie d'une note d'examen : 'Il est impossible de vérifier si la saisie d'un étudiant est valide avant qu'il n'ait tapé une valeur au clavier. On doit donc exécuter la saisie au moins une fois, ce qui fait du do-while la structure parfaitement adaptée.'*

4. La boucle POUR (for) : Itération déterminée

La structure 'for' rationalise la répétition en regroupant au même endroit les trois éléments vitaux : l'initialisation du compteur, la condition de maintien et la mise à jour (incrémentation ou décrémentation). Elle est recommandée lorsque le nombre de répétitions est fixé à l'avance (ex: répéter 50 fois, parcourir un tableau de N éléments). Bien que les trois expressions séparées par des points-virgules soient optionnelles en C, les omettre nuit gravement à la lisibilité et est fortement sanctionné en évaluation.

- En-tête condensé : réunit initialisation, condition de maintien et incrémentation sous la forme `for(init; cond; inc)`.
- Usage privilégié : réservée aux situations où le nombre exact d'itérations est connu avant d'entrer dans la boucle.
- Portée (scope) : possibilité de déclarer la variable d'itération directement au sein de l'en-tête de la boucle.

Cas concret enseignant : *Métaphore de la course sur piste d'athlétisme : 'Vous savez avant de démarrer que vous devez effectuer exactement 5 tours. Vous préparez votre compteur au tour 1 (initialisation), vous vérifiez que vous n'avez pas dépassé 5 (condition), et après chaque franchissement de ligne d'arrivée, vous ajoutez 1 au compteur (incrémentation).'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante ou bloque indéfiniment quand j'oublie d'incrémenter 'i' dans une boucle while ?
Réponse du professeur : Si la variable de contrôle 'i' n'est jamais modifiée dans le corps de la boucle, la condition d'arrêt (ex: i < n) reste indéfiniment vraie. Le programme s'engouffre dans une boucle infinie sans jamais pouvoir sortir, ce qui bloque l'exécution et consomme les ressources du processeur.

Question étudiant : Est-ce qu'on peut toujours remplacer une boucle 'for' par une boucle 'while' ?
Réponse du professeur : Oui, toute boucle 'for' peut être équivalente à une boucle 'while' en plaçant l'initialisation avant la boucle et l'incrémentation à la fin du bloc d'instructions. Cependant, la boucle 'for' doit être préférée dès que le nombre d'itérations est connu d'avance pour des raisons d'expressivité et de clarté du code.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme complet en C qui calcule la somme des N premiers entiers positifs (ex: si N = 5, calculer 1 + 2 + 3 + 4 + 5 = 15). Le programme doit valider que N est un entier strictement positif (> 0) via une boucle 'do-while', puis calculer le cumul au moyen d'une boucle 'for'.

Résolution pas à pas :
Étape 1 : Déclarer les variables entières n, somme (initialisée à 0) et i. Étape 2 : Réaliser la saisie sécurisée avec do { printf("Entrez N (>0) : "); scanf("%d", &n;); } while (n <= 0); afin de rejeter toute valeur négative ou nulle. Étape 3 : Initialiser somme = 0. Étape 4 : Mettre en place la boucle de calcul for (i = 1; i <= n; i++) { somme += i; }. Étape 5 : Afficher le résultat final avec printf.

Résultat : Pour N = 5, la boucle 'for' effectue 5 itérations (i=1 -> somme=1; i=2 -> somme=3; i=3 -> somme=6; i=4 -> somme=10; i=5 -> somme=15). Le programme affiche la valeur finale 15. Si une saisie invalide (<=0) est effectuée au départ, le do-while force la re-saisie.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'évaluation entre une boucle 'while' et une boucle 'do-while' ?

R : La boucle 'while' évalue sa condition en pré-condition (avant l'exécution du bloc, autorisant 0 itération), tandis que 'do-while' l'évalue en post-condition (après l'exécution du bloc, garantissant au moins 1 itération).

Q : Que se passe-t-il si la condition initiale d'une boucle 'for' est fausse dès le premier test ?

R : Le corps de la boucle n'est jamais exécuté (0 itération) et le programme poursuit son exécution à l'instruction située directement après la boucle.

Q : Quel oubli de syntaxe systématique provoque une erreur de compilation sur la boucle 'do-while' en C ?

R : L'oubli du point-virgule final ';' obligatoirement placé après la parenthèse fermante du conditionnel while dans la structure do { ... } while (condition);

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765